Zigzag Traverse

AlgoExpert · Hard · Arrays

The problem

- Read a matrix along anti-diagonals in zigzag, starting at top-left and moving down first (odd diagonals are read bottom-to-top, even ones top-to-bottom).

Intuition

- Walk diagonals, bouncing off the edges and flipping direction each time.

Brute force vs optimal

- Simulate direction changes cell by cell (error-prone)
- Iterate anti-diagonals r + c = d, alternate direction → O(n)
- Each cell once

Approach

- 1. Start at top-left heading down
- 2. Move diagonally
- 3. At an edge, step along it and flip direction
- 4. Stop after every cell

Worked example

1	3	4	10
2	5	9	11
6	8	12	15
7	13	14	16

```
diagonal 0 (down): [1]
```

Diagonal 0

Worked example

1	3	4	10
2	5	9	11
6	8	12	15
7	13	14	16

```
diagonal 1 (up): [2, 3]
```

Diagonal 1

Worked example

1	3	4	10
2	5	9	11
6	8	12	15
7	13	14	16

```
diagonal 2 (down): [4, 5, 6]
```

Diagonal 2

Worked example

1	3	4	10
2	5	9	11
6	8	12	15
7	13	14	16

```
diagonal 3 (up): [7, 8, 9, 10]
```

Diagonal 3

Worked example

1	3	4	10
2	5	9	11
6	8	12	15
7	13	14	16

```
diagonal 5 (up): [14, 15]
```

Diagonal 5

Worked example

1	3	4	10
2	5	9	11
6	8	12	15
7	13	14	16

```
diagonal 6 (down): [16]
```

Diagonal 6

Java solution

```java
public class ZigzagTraverse {
    public static List<Integer> zigzagTraverse(List<List<Integer>> m) {
        List<Integer> out = new ArrayList<>();
        int R = m.size(), C = m.get(0).size();
        for (int d = 0; d < R + C - 1; d++) {
            int lo = Math.max(0, d - C + 1), hi = Math.min(d, R - 1);
            if (d % 2 == 0) for (int r = lo; r <= hi; r++) out.add(m.get(r).get(d - r));
            else for (int r = hi; r >= lo; r--) out.add(m.get(r).get(d - r));
        }
        return out;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Clamp row bounds for each diagonal
- Non-square matrices
- Empty matrix

Interview tips & follow-ups

- Draw the diagonals first

Recap

- Walk diagonals, bouncing off the edges and flipping direction each time.
- Complexity: O(n)